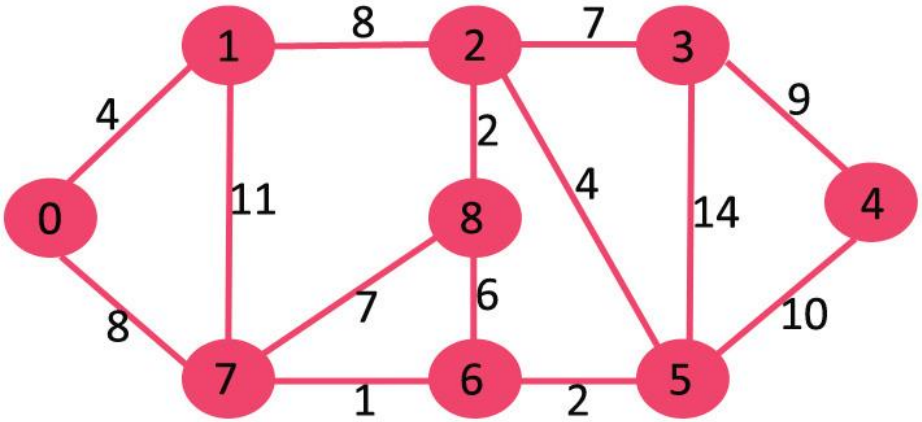
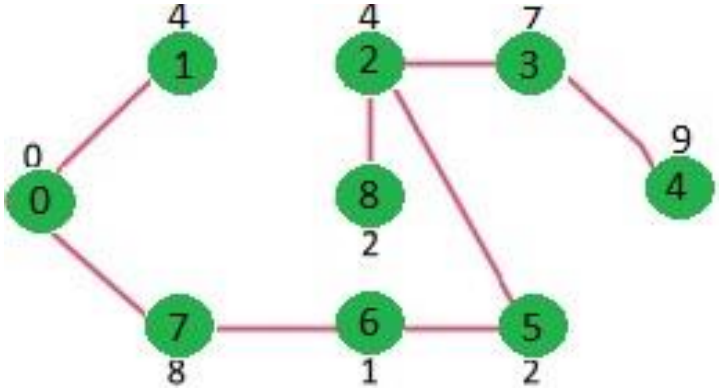
Minimum Spanning Tree

Minimum Spanning Tree

- A **minimum spanning tree** (**MST**) or **minimum weight spanning tree** is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight.





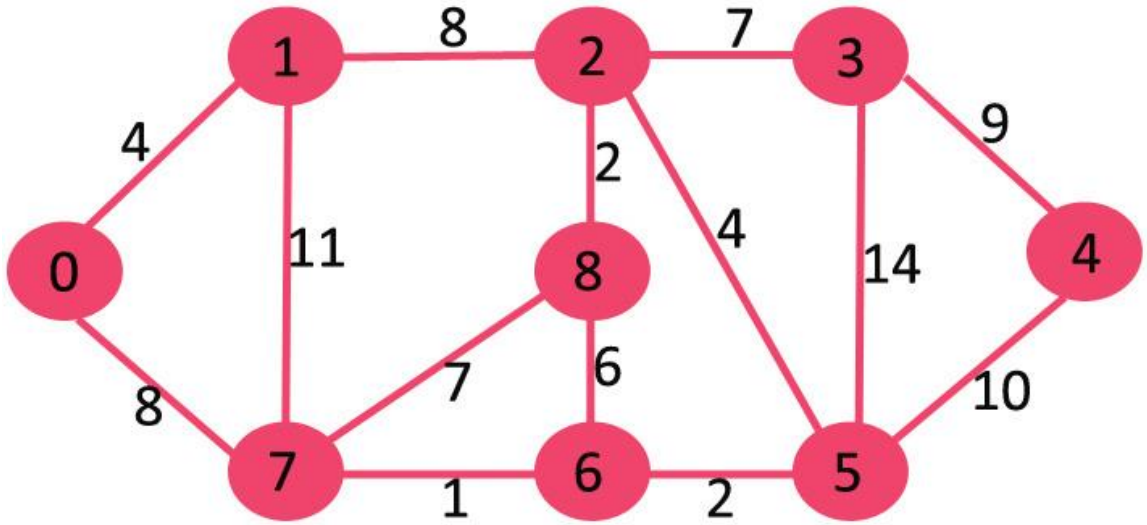
- If there are n vertices in the graph, then each spanning tree has n − 1 edges.
- If the weights are positive, then a minimum spanning tree is in fact a minimum-cost subgraph connecting all vertices.
- For any cycle C in the graph, if the weight of an edge e of C is larger than the individual weights of all other edges of C, then this edge cannot belong to an MST.

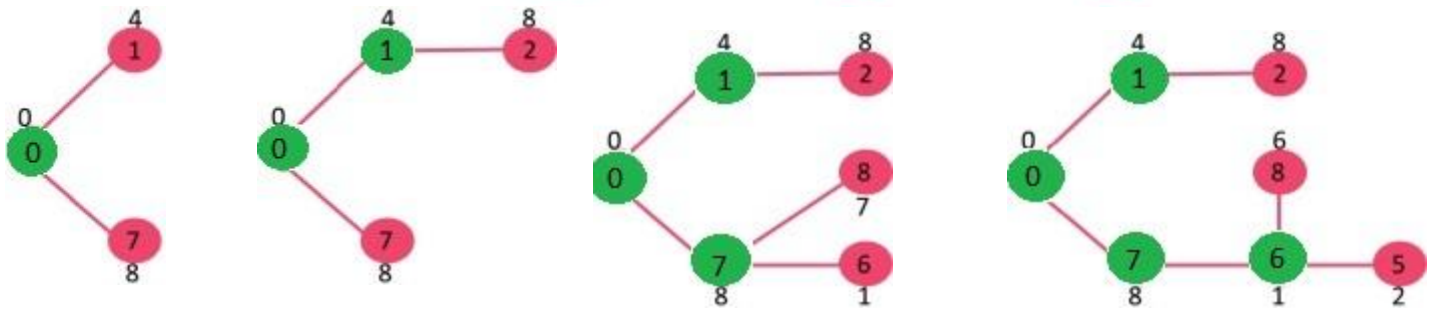
Algorithms

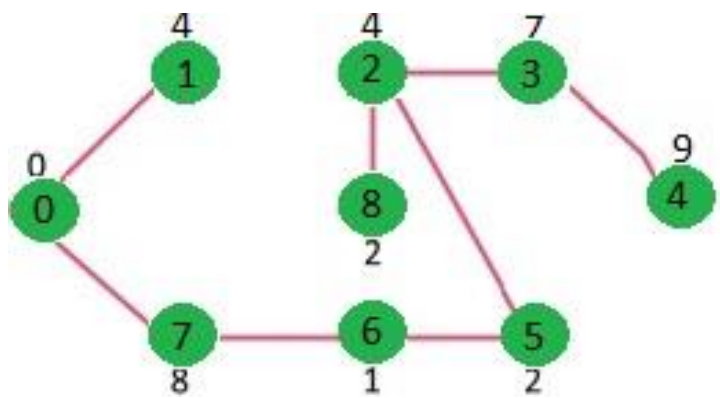
- Borůvka's algorithm-O(E log V) time
- Prim's algorithm-O(E log V) or O((E + V) log V)
- Kruskal's algorithm-O(E log V)
- Reverse-delete algorithm-O(E log V (log log V)^3)

Prim's Algo

Algorithm
1) Create a set *mstSet* that keeps track of vertices already included in MST.
2) Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.
3) While mstSet doesn't include all vertices
....**a)** Pick a vertex *u* which is not there in *mstSet* and has minimum key value.
....**b)** Include *u* to mstSet.
....**c)** Update key value of all adjacent vertices of *u*. To update the key values, iterate through all adjacent vertices. For every adjacent vertex *v*, if weight of edge *u*-*v* is less than the previous key value of *v*, update the key value as weight of *u*-*v*
The idea of using key values is to pick the minimum weight edge from cut. The key values are used only for vertices which are not yet included in MST, the key value for these vertices indicate the minimum weight edges connecting them to the set of vertices included in MST.







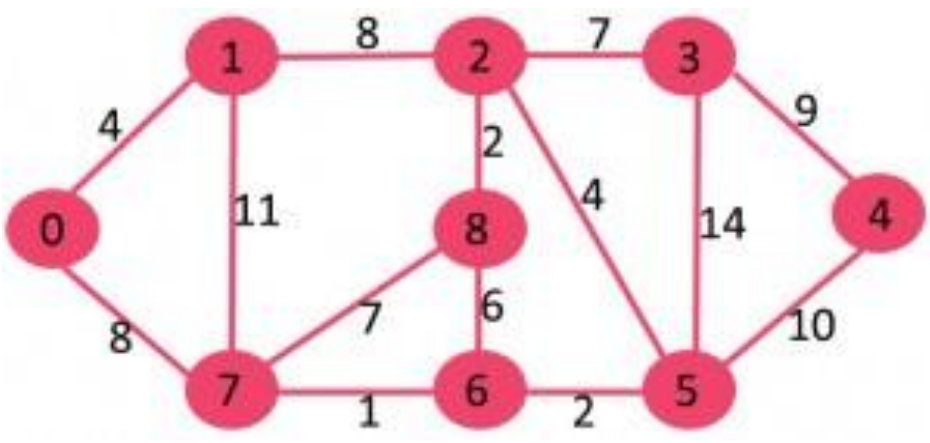
Cost of tree?

```
int graph[V][V] = { {
0, 2, 0, 6, 0 },

   { 2, 0, 3, 8, 5 },

   { 0, 3, 0, 0, 7 },

   { 6, 8, 0, 0, 9 },

   { 0, 5, 7, 9, 0 }
};
```

```
void primMST(int graph[V][V])
{

   int parent[V];
   int key[V];

   bool mstSet[V];


   for (int i = 0; i < V; i++)
      key[i] = INT_MAX, mstSet[i] = false;

     key[0] = 0;
   parent[0] = -1; // First node is always root of MST

   // The MST will have V vertices
   for (int count = 0; count < V - 1; count++) {
       int u = minKey(key, mstSet);

      // Add the picked vertex to the MST Set
      mstSet[u] = true;

           for (int v = 0; v < V; v++)

                 if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v])
            parent[v] = u, key[v] = graph[u][v];
   }

   // print the constructed MST
   printMST(parent, graph);
}
```

Kruskal's Algorithm

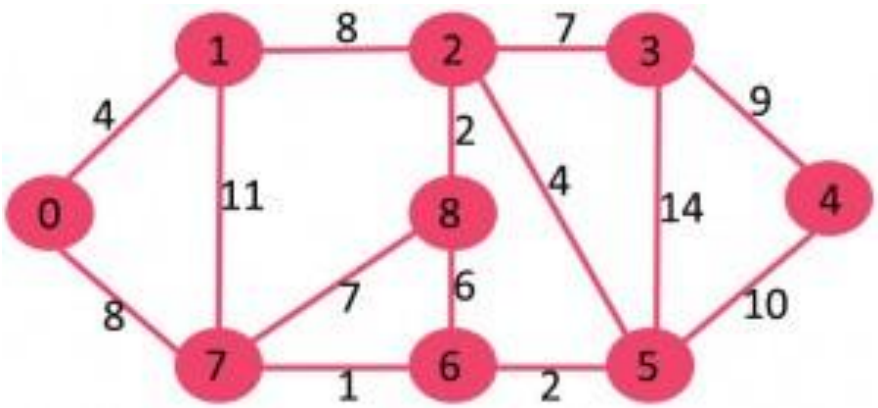
1. *Sort all the edges in non-decreasing order of their weight.*
2. *Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If cycle is not formed, include this edge. Else, discard it.*
3. *Repeat step#2 until there are (V-1) edges in the spanning tree.*

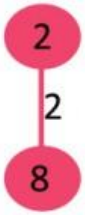


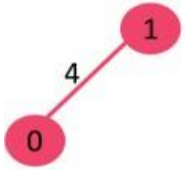
After sorting:

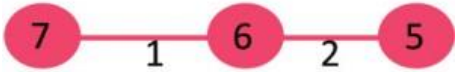
Weight	Src	Dest
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

1. *Pick edge 7-6:* No cycle is formed, include it.
2. *Pick edge 8-2:* No cycle is formed, include it.
3. *Pick edge 6-5:* No cycle is formed, include it.
4. *Pick edge 0-1:* No cycle is formed, include it.

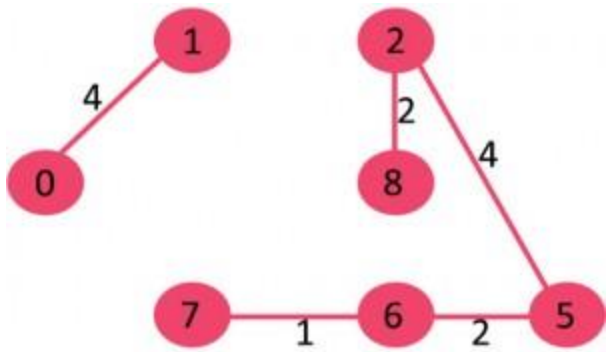




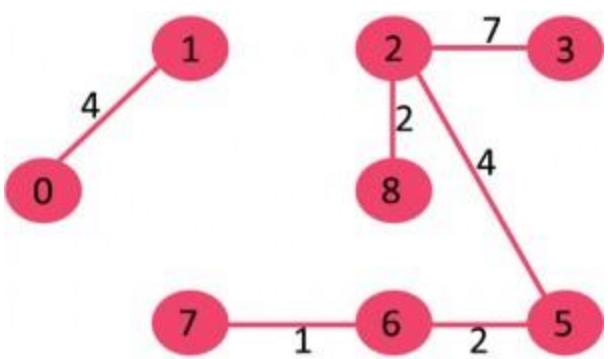




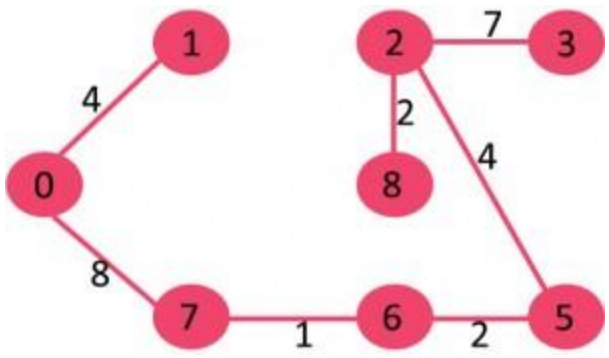
5. *Pick edge 2-5:* No cycle is formed, include it.



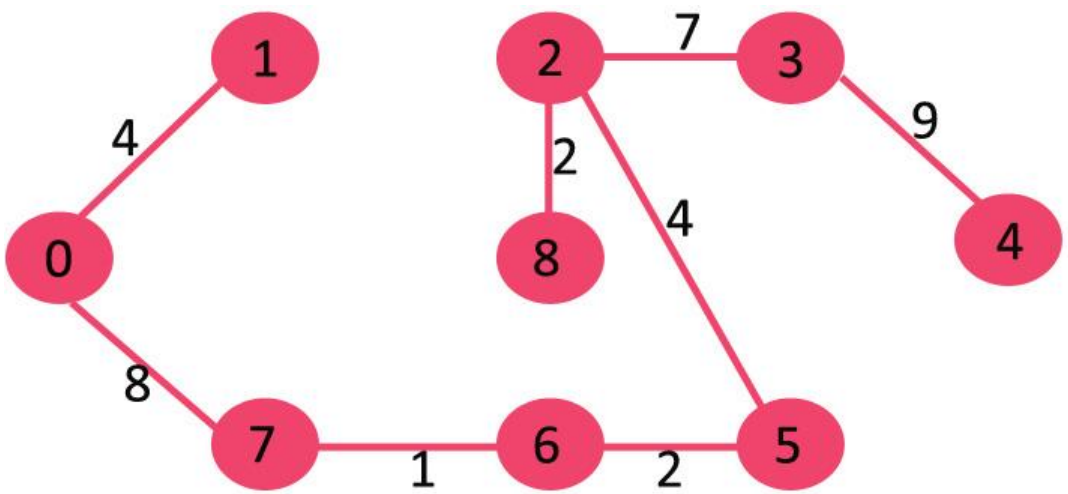
6. *Pick edge 8-6:* Since including this edge results in cycle, discard it.
7. *Pick edge 2-3:* No cycle is formed, include it.



8. *Pick edge 7-8:* Since including this edge results in cycle, discard it.
9. *Pick edge 0-7:* No cycle is formed, include it.



10. *Pick edge 1-2:* Since including this edge results in cycle, discard it.
11. *Pick edge 3-4:* No cycle is formed, include it.



Prim's algorithm is significantly faster in the limit when you've got a really dense graph with many more edges than vertices.

Kruskal performs **better** in typical situations (sparse graphs) because it uses simpler data structures.

- How many different spanning trees are possible?
- How to detect cycle in graph?
- Bipartite Graph?